

Beyond Token Time: Persistent Multi-Stream Latent World States for Modality-Agnostic Intelligence

Position Paper / Research Proposal

Nako Sung · August 2026

Abstract

Modern language models maintain rich continuous hidden states, yet their dominant computational interface repeatedly serializes those states into discrete tokens and advances according to a single token clock. Continuous latent reasoning has shown that this serialization is not always necessary, multi-stream language models have shown that multiple token streams can coexist, and multimodal architectures have shown that heterogeneous observations can share latent representations. We argue that these results motivate a broader architectural hypothesis: intelligence need not be organized around tokens, internal streams need not be linguistic, and computation need not advance according to one global clock.

We propose a **Persistent Multi-Stream Latent World State (PMLWS)**, coordinated by a **Latent Event Bus (LEB)**. Structured latent processes may represent entities, hypotheses, goals, sensory processes, or candidate futures. Each process can be updated selectively and asynchronously when relevant events occur. Text, audio, vision, and actions interact with the shared state through modality-specific adapters rather than defining the substrate of computation. We position this proposal relative to Coconut, Perceiver IO, parallel latent test-time scaling, RL-based latent reasoning, and Multi-Stream LLMs, and define controlled experiments that can falsify the central claims.

1 Introduction

Autoregressive language models are built around a simple rhythm. Given tokens $x_{\leq t}$, a transformer computes a continuous hidden representation h_t , maps it to a vocabulary distribution, selects a discrete symbol, and continues.

continuous state $\rightarrow$ discrete token $\rightarrow$ continuous state

This is natural for language generation. It is less obvious why it should be mandatory for *continuing internal computation*. Coconut directly challenges this assumption by feeding a hidden state back as a subsequent input embedding rather than decoding every intermediate state into a token [1]. But removing intermediate tokens addresses only one restriction. A second remains: computation is still often conceptualized as a single sequence advancing under one global step index.

Real environments are not synchronized this way. A conversational agent may listen, update beliefs, plan, speak, and detect interruption concurrently. A robot may update control at high frequency while planning only when needed. A scene contains many entities whose states evolve independently.

What if the fundamental unit of intelligent computation were not the next token, but the next relevant event?

The proposed contribution is not latent reasoning, parallel streams, multimodality, or reinforcement learning individually. It is the architectural hypothesis that a general agent can maintain **persistent, structured, asynchronous latent processes** whose

updates are triggered by events rather than token production.

2 Modalities as Lenses

Let s denote an underlying world state and let modality m produce an observation $o^m(s) = g_m(s)$. An RGB image, speech waveform, text description, depth map, or motor signal is not the world itself. Each is a particular projection, measurement, or interface.

Shared latent spaces are not new. Perceiver IO demonstrated a general latent-processing architecture for heterogeneous structured inputs and outputs [2]. Our claim is narrower and more operational: the shared latent representation should persist across interactions, evolve recurrently, and contain multiple processes with distinct update schedules.

Modality is an interface to internal state, not necessarily the format of thought.

3 From Token Time to Event Time

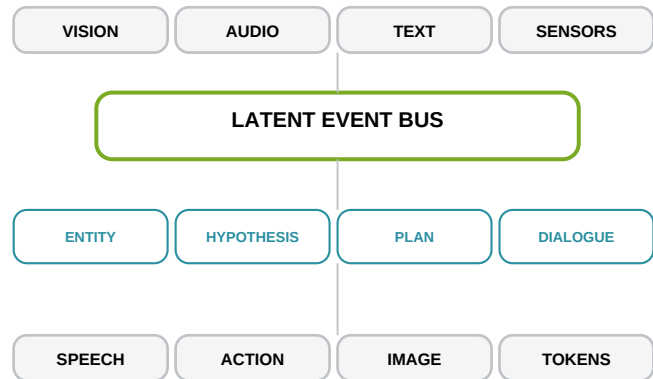
A standard autoregressive model implicitly defines a clock in which each increment is tied to a token position. We call this *token time*. A general agent may instead maintain audio, vision, planning, and motor processes with different natural update schedules.

token time $\neq$ compute time $\neq$ world time

Computation may advance through events $e_1, e_2, \dots$ that activate only relevant state components. A new acoustic observation may update speaker and semantic state; a sudden obstacle may activate vision, planning, and control; if no verbal response is needed, no language token need be produced.

4 Persistent Multi-Stream Latent World State

At event e , let the model maintain $Z_e = [z_e^{(1)}, \dots, z_e^{(K)}]$. Streams may correspond to physical entities, goals, memory elements, candidate hypotheses, predicted futures, perceptual processes, or control processes. An activity mask chooses which streams receive substantial computation at each event; inactive streams persist.



The **Latent Event Bus** is a common representational and routing layer connecting persistent latent streams, observations, and outputs. Modality-specific encoders and decoders do not define the internal reasoning format.

Latent continuation can be one implementation primitive: an internal action can bypass vocabulary decoding and feed a projected hidden state back into computation. This is closely related to Coconut [1] and is not presented as the primary novelty.

5 Delayed Commitment and Concurrent Hypotheses

If several hypotheses remain plausible, they may be maintained as components of one persistent state rather than instantiated as complete independent textual trajectories. Recent work has shown that latent reasoning can also be scaled through parallel stochastic

trajectories and latent reward models [4]. Our claim is therefore not that continuous representations uniquely enable alternatives; it is that unresolved alternatives may remain coordinated components of a persistent world model.

The relevant advantage is not a simplistic claim that a continuous vector contains more information than a token. The defensible claim is that discrete output imposes a particular quantization and semantic commitment, and that delaying this commitment may preserve distinctions useful for later decisions.

6 Relation to Prior Work

Direction	Already established	Difference emphasized here
Coconut [1]	Continuous hidden-state reasoning; latent alternatives.	Persistent state across perception, reasoning, and action.
Perceiver IO [2]	General latent processing across heterogeneous I/O.	Recurrent persistence and selective event-triggered updates.
Hybrid latent RL [3]	RL can optimize latent/discrete reasoning.	RL trains update/scheduling policy; not itself a novelty claim.
Parallel latent TTS [4]	Multiple latent trajectories can be sampled and ranked.	Streams may be entities, goals, modalities, or hypotheses inside one state.
Multi-Stream LLMs [5]	Several token streams can read/write in parallel.	Internal streams need not be tokens or share one update clock.

The intended novelty is the combination of persistence + structured latent multiplicity + asynchronous event-driven scheduling.

7 Training

Stage I - conventional pretraining. Initialize useful representations and modality adapters through language or multimodal predictive training.

Stage II - latent-state bootstrapping. Introduce recurrence, persistent slots, and routing using next-state prediction, entity tracking, masked-state reconstruction, cross-modal prediction, or distillation.

Stage III - outcome-driven optimization. Reinforcement learning is a natural candidate because many internal trajectories may solve a task. Hybrid latent reasoning already establishes RL over latent/discrete policies [3]; here the target policy controls persistent asynchronous state.

A generic objective can reward task success while penalizing unnecessary computation, unnecessary communication/serialization, and unstable recurrent dynamics.

8 Experimental Program

The first useful experiment should be deliberately small. Compare four compute-matched systems: (A) sequential discrete reasoning, (B) parallel token streams, (C) sequential latent recurrence, and (D) asynchronous multi-stream latent state.

8.1 Asynchronous multi-entity world

Construct an environment with independently evolving entities and heterogeneous update rates. Measure entity-state accuracy, retention, cross-stream interference, FLOPs, latency, and memory as entity count increases.

8.2 Delayed evidence

Early observations support several hypotheses; later evidence resolves them. Measure hypothesis diversity, contradiction recovery, and revision cost.

8.3 Interruption

Begin an output or action and inject new information before completion. Measure reaction latency, reusable computation, consistency, and unnecessary recomputation.

8.4 Cross-modal action without text

Provide vision and audio and require a non-linguistic action. Compare explicit text-mediated pipelines against direct

latent-state-to-action behavior.

Killer ablation: if asynchronous structured state does not outperform simpler sequential or parallel baselines under matched compute on tasks designed around independent clocks and interruption, the additional architecture is not justified.

9 Failure Modes and Falsification

Stream collapse. Multiple slots may converge to effectively identical representations. **Fragmentation.** Streams may become too isolated to maintain global consistency. **Latent drift.** Recurrence may push state outside the pretrained distribution. **Scheduling overhead.** Routing may cost more than selective updates save. **False concurrency.** Parallel slots may be cosmetically separate while implementing one serial computation. **Interpretability.** Persistent latent state may be harder to audit than explicit text.

10 Discussion

Language models encourage us to identify communication, reasoning, memory, and scheduling with a single sequence because text was abundant and next-token prediction was scalable. But the architecture through which a capability was discovered need not be the architecture through which it is best organized.

The world does not arrive as a sentence. A room contains many entities simultaneously. Perception need not stop while action occurs. Planning need not stop when new evidence arrives. A hypothesis need not become a sentence before another hypothesis can coexist with it.

Why should language simultaneously serve as interface, reasoning trace, memory representation, scheduler, and computational clock?

11 Conclusion

Continuous latent reasoning shows that models need not verbalize every internal step. Parallel-stream models show that intelligent interaction need not occupy a single message stream. Multimodal architectures show that heterogeneous observations can share latent computation. We propose unifying these observations around a different primitive: **persistent asynchronous state.**

Compute when an event requires computation. Update the state that the event affects. Serialize into a modality only when interaction requires serialization.

Tokens are interfaces. Events move the world.

References

- [1] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. E. Weston, and Y. Tian. Training Large Language Models to Reason in a Continuous Latent Space. ICLR 2025 Workshop on Reasoning and Planning for Large Language Models, 2025.
- [2] A. Jaegle, S. Borgeaud, J.-B. Alayrac, et al. Perceiver IO: A General Architecture for Structured Inputs & Outputs. ICLR, 2022.
- [3] Z. Yue, B. Jin, H. Zeng, et al. Hybrid Latent Reasoning via Reinforcement Learning. NeurIPS, 2025.
- [4] R. You, Y. Li, M. Liu, W. Wang, L. Nie, and W. Li. Parallel Test-Time Scaling for Latent Reasoning Models. ACL, 2026, pp. 44703-44717.
- [5] G. Su, Y. Yang, X. Li, and J. Geiping. Multi-Stream LLMs: Unblocking Language Models with Parallel Streams of Thoughts, Inputs and Outputs. arXiv:2605.12460, 2026.